Contents

ClaudIU user manual

ClaudIU (working codename) is a local, browser-based client for AI coding agents that speak the Agent Client Protocol (ACP). It runs a small Python server on your machine, spawns the agent's ACP adapter as a plain subprocess, and renders the structured conversation — messages, thinking, tool calls with diffs, permission requests — in your browser. Nothing is screen-scraped; everything you see comes from typed protocol data, and every byte that crosses the wire is recorded.

Starting

```
python -m claudiu
```

Options: `--port N` (default 0 = OS-assigned), `--profiles DIR` (default `agents/`), `--records DIR` (default `~/.claudiu/records`), `--records-keep-days N` (delete records older than N days at startup; default 0 = keep them all), `--no-drift-online` (skip the update check), `--token-file FILE` (keep the auth token across launches). The server prints a URL with a token — open it; the token becomes a cookie and the address bar cleans itself. By default each launch gets a fresh token and old URLs die with the server; with `--port` and `--token-file` together the URL is stable and can be bookmarked (the file is created owner-only where the OS supports it — treat it like a password).

Tabs

Each session is a tab (Alt+1…9 switch, Alt+N opens the launcher, × closes and ends the session). The tab shows the agent's own title for the session once it sets one, and a dot: green ready, pulsing blue working, red failed or disconnected. The status strip, selectors and composer always belong to the active tab; approval dialogs from any tab pop up wherever you are, labelled with the session they belong to.

The launcher

Pick an **agent** (from `agents/*.toml` profiles; an agent whose adapter is not installed shows "adapter missing" with the install hint) and a **project directory** — the session's working directory and, importantly, its *file-access boundary*: the agent can only read/write inside it through this client. Type the path or click **Browse...**: a folder picker lists the subdirectories of the current path (drive roots are one click away, ↑ **Up** goes to the parent) and **Use this folder** fills the field; the browser remembers the last directory a session was started in. Known caveats of the selected agent are listed right there, followed by the **runtime** the adapter will be pointed at: for Claude, `CLAUDE_CODE_EXECUTABLE` resolves to your installed `claude` when there is one on `PATH`. The adapter bundles its own, older Claude CLI, which the API may refuse for newer models ("version 2.1.251 or newer is required" was the 2026-09-01 symptom); with no `claude` installed the bundled copy is used. Adapter diagnostics (one

`[session/query] …` line per session) arrive on stderr and are counted in the **log** chip of the status strip. **Find resumable sessions** asks the agent which of its own sessions exist for that directory (a throwaway adapter is started and closed for the query) and offers a **Resume** button per session. Resuming replays the conversation so far — prompts, answers, tool calls — before the composer enables (for the rare agent that can only attach without history, the pane starts empty).

The conversation

- **Agent text** streams as it is produced; *thinking* appears dimmed and italic; your prompts are boxed. Thinking is the model's own summary of its reasoning — the API offers summaries or nothing for recent models — and a turn with only a `· thinking ·` marker had no summary to send.
- **Tool calls** are one row per call, updated in place as the agent reports progress (pending → in progress → completed/failed, colour-coded edge). Edits arrive as real diffs (+/− lines); text output and file locations open under "details".
- **Hierarchy, like the terminal**: text the agent produces *before* a tool call is a step and renders attenuated; the final answer is bright. Tool rows are dim with a status dot (blue pending, green done, red failed).
- **Markdown-lite**: `**bold**` is highlighted in the accent colour, `` `code` `` and fenced blocks are monospaced, headings stand out. The text is rendered as DOM nodes, never as HTML.
- **Tool output is collapsed by default** (as in the terminal); the "expand tool output" checkbox in the toolbar opens every result and is remembered by the browser; each row's "details" toggle always works.
- **Plan** entries (the agent's todo list) fill the panel above the composer.
- **Turn ends** are marked with the protocol's stop reason.
- The `{}` button on special rows reveals the raw protocol frame behind them — the escape hatch into the session record.

Prompting

Enter sends; Shift+Enter inserts a newline. The Send button is enabled only while the session is ready (not during the agent's turn, not before startup completes). **Stop** cancels the current turn. Typing `/` first opens the command palette listing the agent's own slash commands; picking one only fills the composer — nothing is sent until you press Send, and the agent's response to a command is always rendered.

Claude Code's *prompt suggestions* (the predicted next prompt the terminal offers after a turn) are not available here yet: the Agent SDK provides them, but the ACP adapter neither enables the option nor forwards the message. The launcher lists this under the agent's caveats.

What the conversation shows — the five lanes

The status strip carries five switches: **thinking**, **tools**, **subagents**, **events**, **harness**. Switching one off removes those rows from the page entirely (they are still recorded, and switching it back on brings them back); the agent's answers and your own prompts are never hidden. The choice is remembered per browser.

Thinking is the one lane that is more than a view filter: what the agent is *asked* to produce is chosen in the launcher (**Thinking**: summarized, omitted, or off) because ACP fixes it when the session is created. Choosing *off* means no thinking tokens are requested at all for that session.

The conversation follows new rows only while you are at the bottom. Scroll up to read and it stays where you put it; a **↓ jump to latest** button appears and puts you back in the stream.

Keeping your sessions

Sessions live in the server, not the page: reloading the browser (or opening a second window) reattaches to everything still running, with the conversation replayed, and lands you back in the tab you were in.

Only one attachment per agent session is allowed. Resuming an agent session that is already open somewhere is refused, naming the session that holds it — two adapters attached to one agent session would write the same transcript file and corrupt it.

Archiving a conversation

Archive in the toolbar hands the session to claude-session-publisher, which writes it into your usual archive directory (`CLAUDE_ARCHIVE_DIR`) in the formats it always produces. The server must know where that tool is: start it with `--archiver <path to transcript_archiver.py>` or set `CLAUDIU_ARCHIVER`. Without it the button says so; nothing is copied into this project.

The top right: account limits, help, settings

The agent reports your **account's** rate-limit windows as it works — the five-hour window, the seven-day one, per-model windows, and whether extra credits are in use. Each window the agent has mentioned appears as a chip at the top right with how much of it is used; hovering gives the status and when it resets. Windows nobody reported are not shown: this view never invents a number it was not told.

? opens a short guide to everything on screen. holds the settings that belong to the browser rather than to a session: the **theme** (follow the system, dark, or light) and whether tool output starts open. Session options — which agent, which directory, what thinking to ask for — are chosen per session in the launcher (the + tab).

The **log** chip is the adapter's own diagnostics. The **anomalies** and **drift** chips are things this client did not expect; clicking one opens a drawer with every entry, kept until you press **dismiss all**. Looking at them never throws them away.

Approvals

When the agent wants to do something that needs permission, a dialog shows the tool call (with diff when provided) and the agent's own options — allow once, allow always, reject — as buttons (digits 1–9 work too). There is no Escape-to-dismiss: an explicit choice is required. The options and their number come from the agent: Claude offers a standing "Always Allow" for some tools only, so a third button appears only when the agent offers a third option. Standing grants are listed after the one-shot choices and drawn dashed/amber; they are never disabled. Unanswered requests are auto-rejected after a timeout (default one hour) and marked as fail-safe rejections.

Questions from the agent

The agent can ask you a multiple-choice question instead of guessing (its AskUserQuestion tool). It arrives as a form: single-answer questions are radio buttons, multi-answer ones checkboxes, and every question has an **Other** box for an answer in your own words, which wins over the options. **Skip** answers nothing — the agent is told you skipped and carries on; the same happens by itself if the question goes unanswered past the timeout. Your answer is written into the conversation so the transcript shows what you chose. An option's *preview* (the mockup the terminal shows on focus) is not displayed here; its description is.

Status strip and warnings

The strip shows the session state, the current permission mode, a **context gauge** (tokens used / window size, percentage, and cost when the agent reports it) and the current model. The toolbar under the prompt holds the selectors: **mode**, **model** and whatever other **configuration options** the agent advertises (Claude Agent 0.73 offers mode, model, effort and agent). Each thing appears once — a config option replaces the legacy select for the same thing — and long descriptions are tooltips. While a turn runs, a pulsing "agent working… 42s · last activity 3s ago (tool_call: Edit hello.py)" row sits at the end of the conversation — the elapsed time tells you the turn is alive, the last-activity part tells you whether the agent is still producing events (it turns amber after a minute of silence). Three chips can appear; the first two should not be ignored:

- **drift** — the agent sent protocol data newer than this client's pinned ACP schema (hover for details). The client keeps working and keeps recording; the flag means an update to the client is due.
- **anomalies** — something out of order happened (malformed frame, adapter crash, rejected command…); the conversation shows the details inline with raw-frame access.

- **log (n)** — the adapter's own log (its stderr: one `[session/query] …` line per session, sometimes echoed command output). These are diagnostics, not errors; real problems come as anomalies. Click to open the drawer under the strip. Never shown inline, never dropped — it is in the record.

Records

If the browser loses its connection — the server restarted, the machine slept — the tab reconnects on its own and asks only for what it missed; the session itself never stopped, because it lives in the server, not the page. While it is trying, the status strip says *reconnecting*. Two things it will not retry: a refused login and a session the server no longer has, both of which say so and ask you to reload.

Every session appends to `<records-dir>/<session>.jsonl`: each protocol frame verbatim (before any interpretation) plus every client action — prompts, permission answers, file-access decisions with the policy that made them. This is the audit trail and the ground truth; the UI's `raw_ref` numbers index into it.

Security notes

The server binds 127.0.0.1 only; every request needs the launch token; a Host that is not loopback, or an Origin that is not this server's own address and port, is refused (cookies are not scoped by port, so a page served from another local port would otherwise arrive authenticated). Agent file access is confined to the project directory (symlinks resolved). Adapter subprocesses run with a scrubbed environment (plus the profile's declared runtime resolution and settings, written as the first record of the session) and die with their session. The record files never contain your token — but they do contain your conversation, so treat the records directory accordingly.